

Report Esercizi Shell

Kali Linux — Comandi da terminale e gestione del filesystem

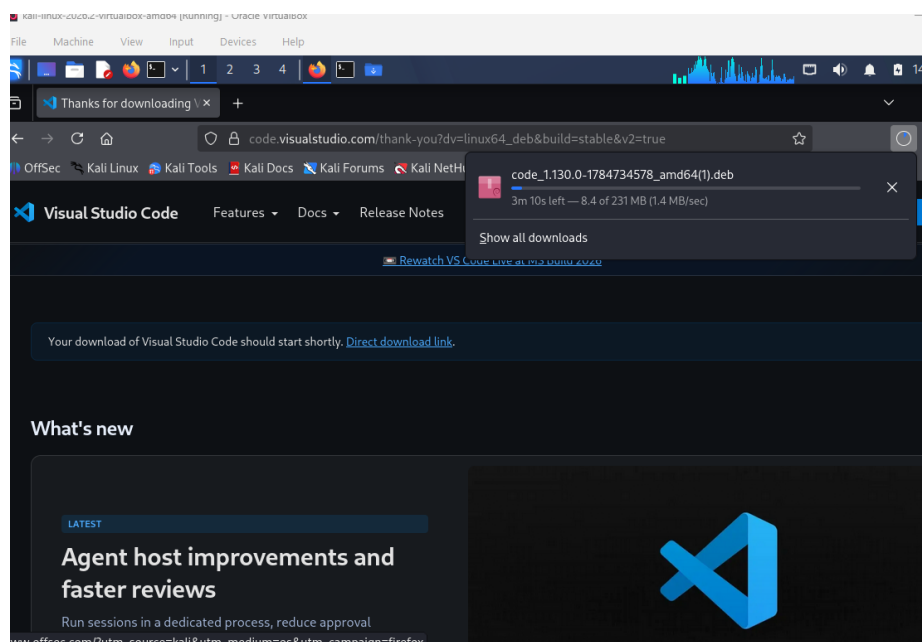
Questo report documenta, passo per passo e con screenshot alla mano, l'installazione dell'ambiente di lavoro e lo svolgimento dell'Esercizio 1 sui comandi shell (`cd`, `mkdir`, `rmdir`, `mv`, `cp`, `rm`, `ls`, `pwd`, `chmod`) su Kali Linux. Per ogni passaggio viene spiegato cosa si voleva ottenere, quale comando è stato usato e perché, e cosa mostra lo screenshot corrispondente.

Installazione di Visual Studio Code

Prima di iniziare gli esercizi veri e propri, è stato installato un editor di testo (Visual Studio Code) su Kali Linux, utile per scrivere e modificare comodamente i file usati più avanti. L'installazione su sistemi Debian-based come Kali avviene scaricando il pacchetto `.deb` dal sito ufficiale e installandolo con `dpkg`, il gestore di pacchetti di basso livello di Debian.

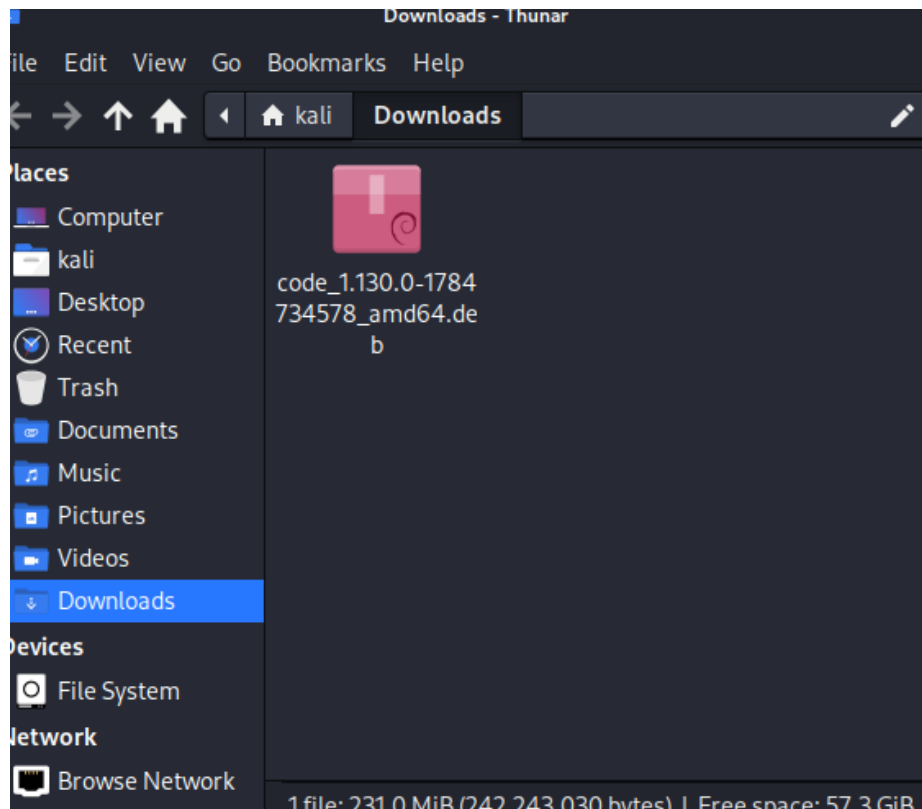
Download del pacchetto

Dal browser Firefox è stata aperta la pagina ufficiale `code.visualstudio.com` e scelto il download della versione `.deb` a 64 bit, quella corretta per Kali Linux (basato su Debian, architettura `amd64`).



Pagina di download di Visual Studio Code (code.visualstudio.com): il download del file .deb per Linux è avviato.

Una volta completato il download, il file è stato verificato all'interno della cartella Downloads, dove il browser salva di default i file scaricati.



Installazione del pacchetto

Per installare un pacchetto .deb da terminale si usa il comando dpkg con l'opzione -i (install), anteposto da sudo perché l'installazione di software richiede i privilegi di amministratore:

```
sudo dpkg -i code_1.130.0-1784734578_amd64.deb
```

Il sistema chiede la password dell'utente kali (necessaria per operazioni con sudo), quindi procede a spaccettare (unpacking) e configurare (setting up) il pacchetto.

```
kali@kali: ~/Downloads
on Actions Edit View Help
kali@kali)-[~/Downloads]
sudo dpkg -i code_1.130.0-1784734578_amd64.deb
[ password for kali:
unpacking previously unselected package code.
unpacking database... 425756 files and directories currently installed.)
unpacking to unpack code_1.130.0-1784734578_amd64.deb...
unpacking code (1.130.0-1784734578)...
unpacking up code (1.130.0-1784734578)...
unpacking triggers for desktop-file-utils (0.28-1)...
unpacking triggers for mailcap (3.76)...
unpacking triggers for shared-mime-info (2.4-5+b3)...

kali@kali)-[~/Downloads]
sudo apt update && sudo apt install gdb
and 'gsudo' not found, did you mean:
command 'sudo' from deb sudo
command 'gfsudo' from deb gfarm-client
sudo apt install <deb name>

kali@kali)-[~/Downloads]

kali@kali)-[~/Downloads]
sudo apt update && sudo apt install gdb
https://packages.microsoft.com/repos/code stable InRelease [3,590 B]
https://packages.microsoft.com/repos/code stable/main amd64 Packages [26.5 kB]
http://kali.download/kali kali-rolling InRelease [34.0 kB]
http://kali.download/kali kali-rolling/main amd64 Packages [21.3 MB]
http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.1 MB]
5 Contents-amd64 11.7 MB/53.1 MB 22% 1,481 kB/s 28s
```

Avvio dell'installazione: viene richiesta la password, poi il pacchetto viene spaccettato e si tenta un aggiornamento dei repository.

```
kali@kali: ~/Downloads
on Actions Edit View Help
kali@kali)-[~/Downloads]
sudo dpkg -i code_1.130.0-1784734578_amd64.deb
[ password for kali:
unpacking previously unselected package code.
unpacking database... 425756 files and directories currently installed.)
unpacking to unpack code_1.130.0-1784734578_amd64.deb...
unpacking code (1.130.0-1784734578)...
unpacking up code (1.130.0-1784734578)...
unpacking triggers for desktop-file-utils (0.28-1)...
unpacking triggers for mailcap (3.76)...
unpacking triggers for shared-mime-info (2.4-5+b3)...

kali@kali)-[~/Downloads]

```

L'installazione risulta completata correttamente: si vedono i messaggi "Unpacking code" e "Setting up code" senza errori.

Aggiornamento del sistema e installazione di gdb

Dopo l'installazione di VS Code, è stato eseguito un aggiornamento dei repository e installato anche il debugger gdb, utile per lo sviluppo:

sudo apt update && sudo apt install gdb

Il comando apt update scarica l'elenco aggiornato dei pacchetti disponibili (compreso ora il repository di Microsoft aggiunto insieme a VS Code), mentre apt install gdb scarica e installa gdb con le sue dipendenze.

```
(kali@kali)-[~/Downloads]
$ sudo apt update && sudo apt install gdb
Get:1 https://packages.microsoft.com/repos/code stable InRelease [3,590 B]
Get:2 https://packages.microsoft.com/repos/code stable/main amd64 Packages [26.5 kB]
Get:3 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:4 http://kali.download/kali kali-rolling/main amd64 Packages [21.3 MB]
Get:5 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.1 MB]
Get:6 http://kali.download/kali kali-rolling/contrib amd64 Packages [103 kB]
Get:7 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [189 kB]
Get:8 http://kali.download/kali kali-rolling/non-free amd64 Packages [169 kB]
Get:9 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [888 kB]
Get:10 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [15.8 kB]
Get:11 http://kali.download/kali kali-rolling/non-free-firmware amd64 Contents (deb) [40.5 kB]
Fetched 75.9 MB in 1min 5s (1,168 kB/s)
808 packages can be upgraded. Run 'apt list --upgradable' to see them.
Upgrading:
  python3.14-tk

Installing:
  gdb

Installing dependencies:
  libdebuginfod-common  libpython3.14  libsource-highlight-common
  libdebuginfod1t64    libpython3.14-minimal  libsource-highlight4t64
  libipt2              libpython3.14-stdlib

Suggested packages:
  gdb-doc  gdbserver  libc-dbg

Summary:
  Upgrading: 1, Installing: 9, Removing: 0, Not Upgrading: 807
  Download size: 10.2 MB
  Space needed: 39.1 MB / 61.3 GB available

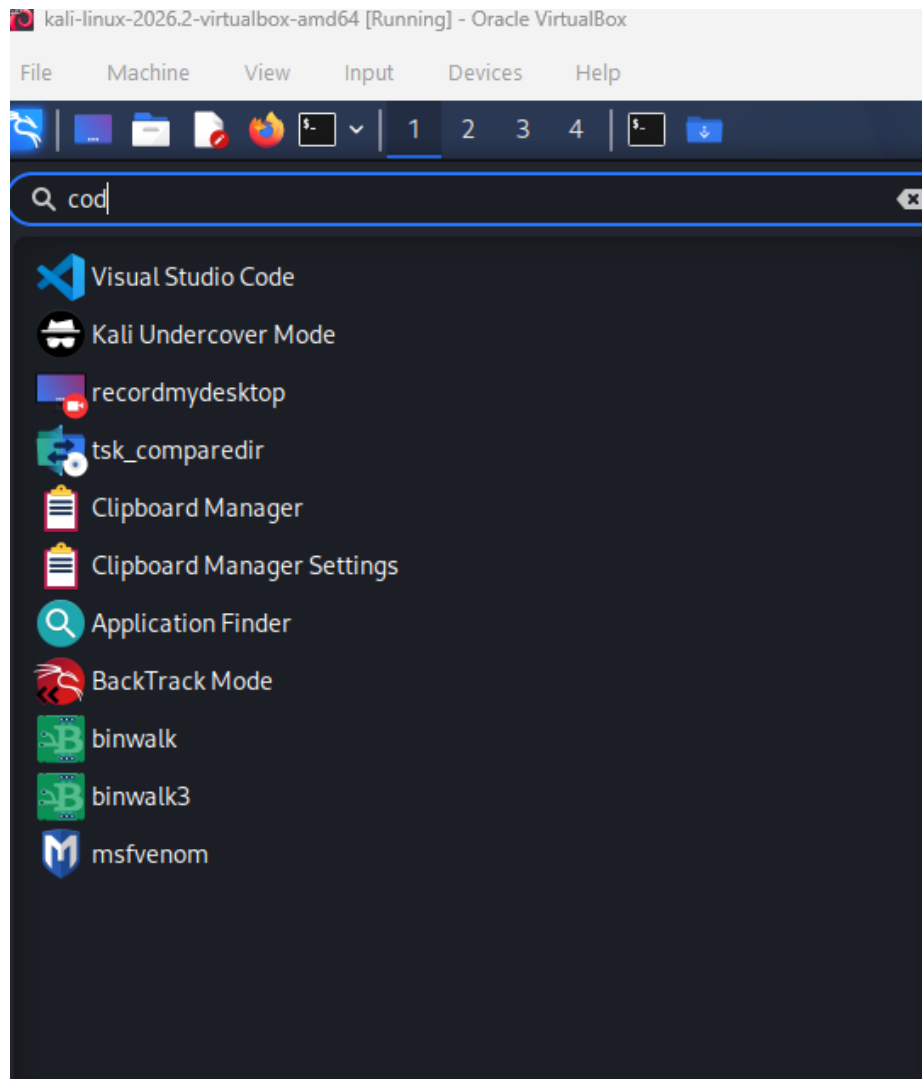
Continue? [Y/n] Y
Get:2 http://kali.download/kali kali-rolling/main amd64 libdebuginfod1t64 amd64 0.195-1 [29.5 kB]
Get:1 http://kali-mirror.garr.it/kali kali-rolling/main amd64 libdebuginfod-common all 0.19
```

Output completo di sudo apt update && sudo apt install gdb: i repository (incluso quello di VS Code) vengono aggiornati e gdb viene installato con le sue dipendenze.

Verifica dell'installazione

Per verificare che VS Code fosse installato correttamente, è stata aperta la ricerca applicazioni del sistema digitando "cod": l'icona di Visual Studio Code

compare tra i risultati, a conferma che l'installazione è andata a buon fine ed è disponibile nel menu applicazioni.



Ricerca "cod" nel menu applicazioni: Visual Studio Code compare correttamente tra i risultati disponibili.

Creazione della struttura di cartelle

Il primo esercizio richiede di ricreare, a partire dalla propria HOME, l'albero di cartelle mostrato nella traccia (dos, studenti con le sottocartelle nicola/anna/matteo e i relativi sotto-alberi, windows, tmp). Per ogni cartella è stato usato il comando mkdir (make directory), spostandosi di volta in volta nella cartella padre corretta con cd.

Come prima cosa è stato controllato il contenuto già presente nella HOME con ls, poi sono state create le quattro cartelle di primo livello richieste dalla traccia:

ls

mkdir dos studenti windows tmp

Passando più argomenti a mkdir sulla stessa riga si creano più cartelle contemporaneamente, tutte all'interno della directory corrente.

```
Session  Actions  Edit  View  Help
(kali㉿kali)-[~]
$ ls
desktop  Documents  Downloads  Music  Pictures  Projects  Public  Templates  Videos
(kali㉿kali)-[~]
$ mkdir dos studenti windows tmp
```

Contenuto iniziale della HOME (ls) e creazione delle quattro cartelle di primo livello con mkdir dos studenti windows tmp.

Entrando in studenti sono state create le tre sottocartelle nicola, anna e matteo, dopodiché ci si è spostati dentro nicola per proseguire con il sotto-albero più "profondo" richiesto dalla traccia:

cd studenti

mkdir nicola anna matteo

cd nicola

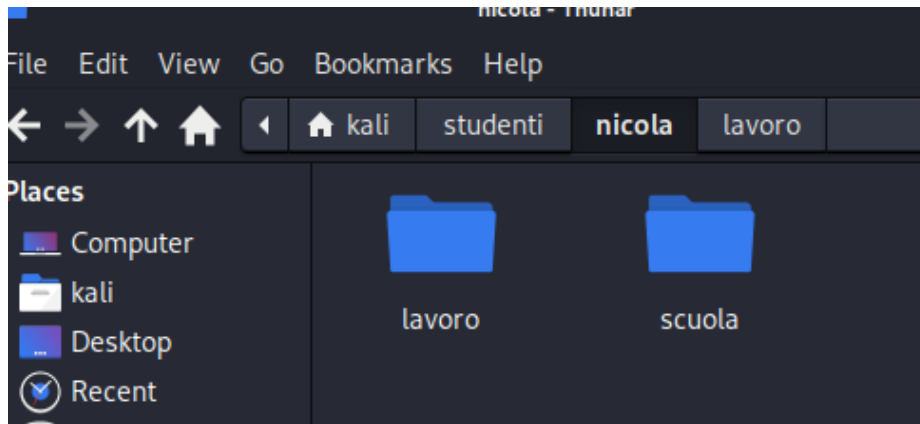
```
(kali㉿kali)-[~]
$ cd studenti

(kali㉿kali)-[~/studenti]
$ mkdir nicola anna matteo

(kali㉿kali)-[~/studenti]
$ cd nicola
```

Ingresso in studenti, creazione delle sottocartelle nicola, anna, matteo, e spostamento dentro nicola per continuare la struttura.

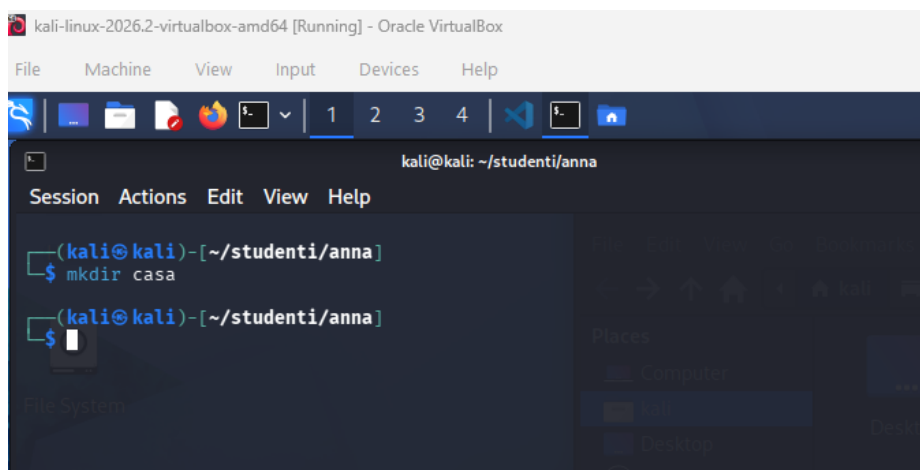
All'interno di nicola sono state create le due sottocartelle scuola e lavoro (mkdir scuola lavoro); il risultato è verificabile subito sotto tramite il file manager grafico Thunar:



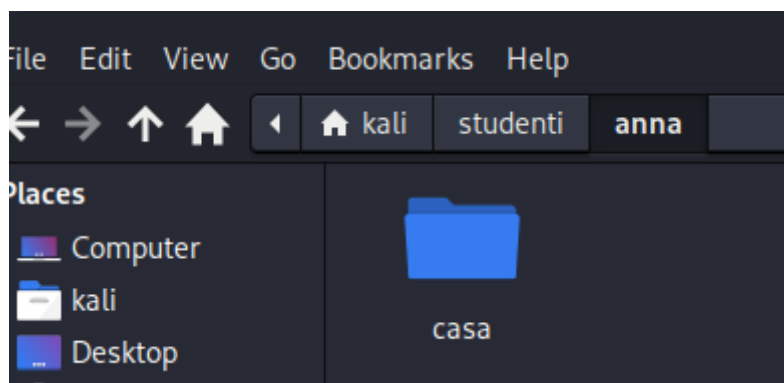
Verifica visuale in Thunar: all'interno di nicola sono effettivamente presenti le cartelle lavoro e scuola, come richiesto dallo schema dell'esercizio.

Tornati in studenti e poi entrati in anna, è stata creata la cartella casa:

```
cd ~/studenti/anna
mkdir casa
```

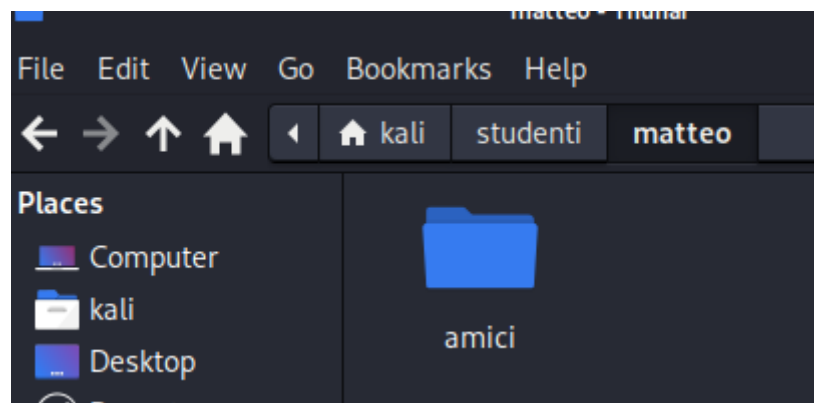


Creazione della cartella casa dentro anna con il comando mkdir casa.



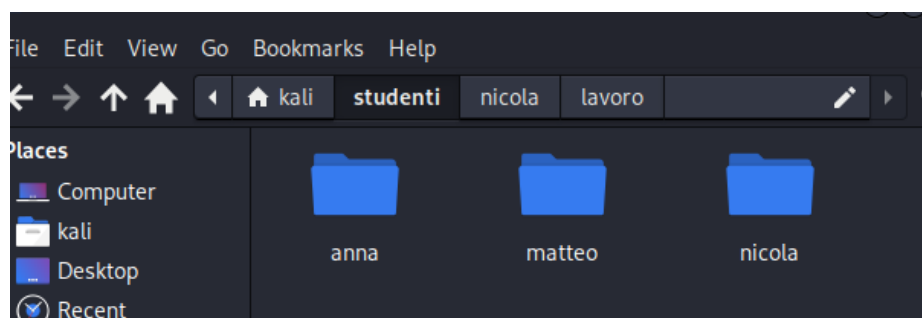
Verifica in Thunar: la cartella casa è correttamente presente dentro anna.

Allo stesso modo, dentro matteo è stata creata la cartella amici (mkdir amici):



Verifica in Thunar: la cartella amici è correttamente presente dentro matteo.

Una volta completato il sotto-albero, uno sguardo d'insieme su studenti conferma che tutte e tre le sottocartelle (anna, matteo, nicola) sono state create correttamente:



Vista d'insieme della cartella studenti: sono presenti tutte e tre le sottocartelle previste, anna, matteo e nicola.

Infine, tornati nella HOME, il comando ls -r (che elenca il contenuto in ordine alfabetico inverso) conferma che tutte le cartelle di primo livello richieste dalla traccia sono state create, insieme a quelle di default già presenti nel sistema:

```
(kali㉿kali)-[~]
$ cd ../../anna/casa
cd: no such file or directory: ../../anna/casa

(kali㉿kali)-[~]
$ ~/studenti/nicola/lavoro

(kali㉿kali)-[~/studenti/nicola/lavoro]
$ ~/studenti/anna/casa

(kali㉿kali)-[~/studenti/anna/casa]
$ cd ../../anna/casa

(kali㉿kali)-[~/studenti/anna/casa]
$
```

Verifica finale con `ls -r` dalla HOME: risultano presenti `dos`, `studenti`, `windows` e `tmp`, oltre alle cartelle di default del sistema.

Da lavoro (sotto nicola) a casa (sotto anna)

Trovandosi nella directory lavoro (sotto nicola), viene richiesto di raggiungere la directory casa (sotto anna) sia con un percorso relativo sia con un percorso assoluto.

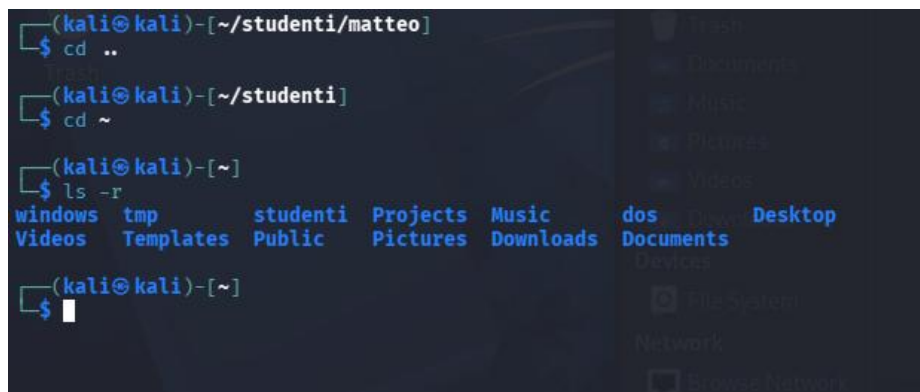
Percorso relativo — partendo da `.../nicola/lavoro`, si risale di due livelli (prima a nicola, poi a studenti) e poi si scende in `anna/casa`:

```
cd ../../anna/casa
```

Percorso assoluto — indipendentemente da dove ci si trova, si specifica il percorso completo a partire dalla HOME (simbolo `~`):

```
cd ~/studenti/anna/casa
```

Nello screenshot si vede un primo tentativo di percorso relativo fallito, eseguito però dalla posizione sbagliata (non da lavoro): l'errore "no such file or directory" mostra bene come, con un percorso relativo, il risultato dipenda sempre dalla directory in cui ci si trova al momento del comando. Subito dopo lo spostamento riesce correttamente, sia usando il percorso assoluto sia, una volta nella posizione giusta, quello relativo:



```
(kali@kali)~[/studenti/matteo]
$ cd ..

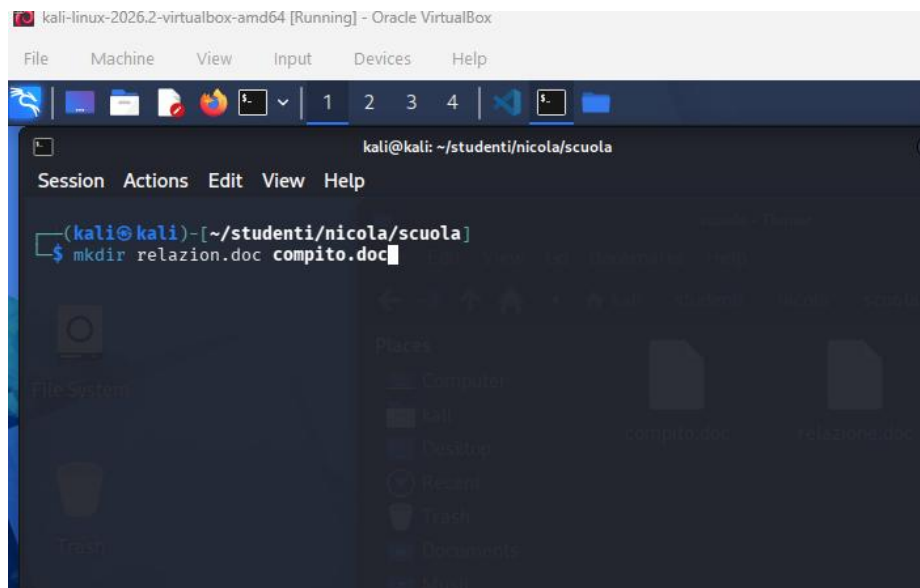
(kali@kali)~[/studenti]
$ cd ~

(kali@kali)~[~]
$ ls -r
windows tmp      studenti Projects Music  dos  Desktop
Videos  Templates Public  Pictures Downloads Documents
(kali@kali)~[~]
$
```

Un primo `cd ../../anna/casa` fallisce perché eseguito dalla posizione sbagliata; lo spostamento riesce poi correttamente sia con il percorso assoluto sia, dalla directory di partenza corretta, con quello relativo.

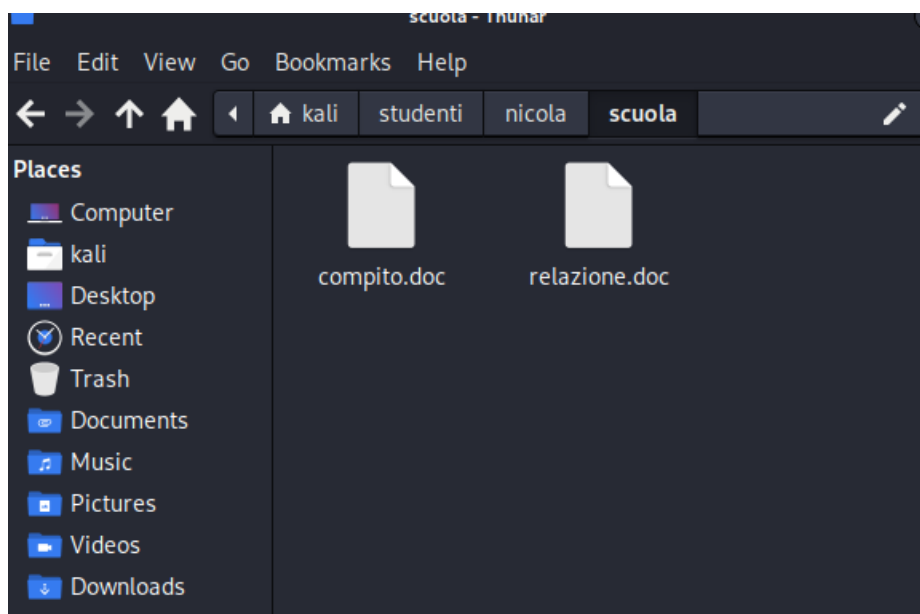
Copiare compito.doc da scuola a casa

Prima di poter copiare i file, sono stati creati i due file richiesti dalla traccia (`relazione.doc` e `compito.doc`) all'interno della cartella scuola, così da avere del contenuto reale con cui lavorare nei passaggi successivi:



Creazione dei file relazione.doc e compito.doc all'interno della cartella scuola.

Una verifica con il file manager conferma che entrambi i file sono presenti nella cartella scuola:



Verifica in Thunar: i file compito.doc e relazione.doc sono presenti nella cartella scuola.

A questo punto, per copiare (non spostare) il file compito.doc dalla cartella scuola alla cartella corrente (casa), è stato usato cp, che duplica il file lasciando l'originale intatto; il simbolo. (punto) indica "qui, nella directory corrente":

cp ../../nicola/scuola/compito.doc .

Nello screenshot un primo cd ../scuola fallisce (percorso relativo non corretto rispetto alla posizione di partenza in quel momento), dopodiché il file viene

copiato correttamente con `cp compito.doc ../../anna/casa`, eseguito questa volta dalla cartella scuola verso la cartella casa:

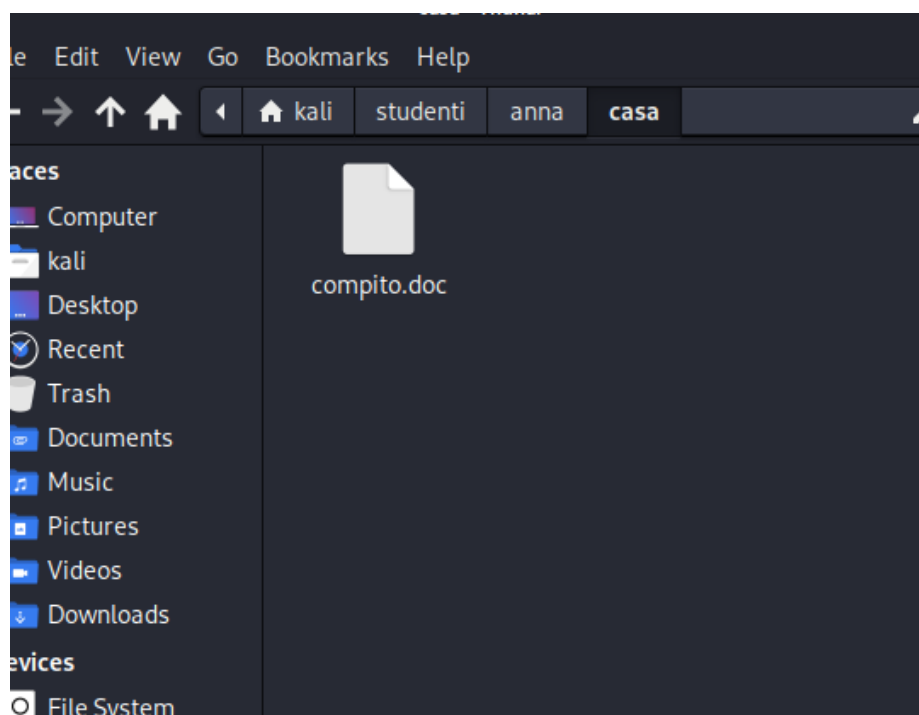
```
(kali㉿kali)-[~/studenti/nicola/scuola]
$ cd ../scuola
zsh: no such file or directory: cd ../scuola

(kali㉿kali)-[~/studenti/nicola/scuola]
$ cp compito.doc ../../anna/casa

(kali㉿kali)-[~/studenti/nicola/scuola]
$
```

Un primo `cd ../scuola` non riesce; il file `compito.doc` viene poi copiato correttamente verso `.../anna/casa` con `cp`.

Il risultato viene verificato aprendo la cartella casa: il file `compito.doc` risulta ora presente anche lì, mentre l'originale resta invariato in scuola, proprio perché `cp` copia senza rimuovere la sorgente:



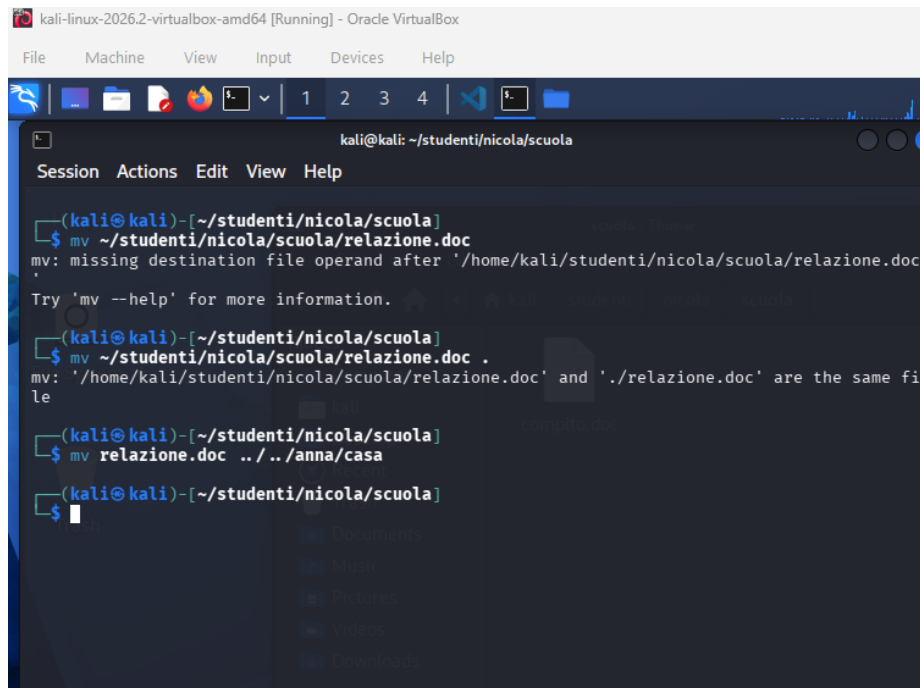
Verifica in Thunar: `compito.doc` è stato copiato correttamente nella cartella casa.

Spostare `relazione.doc` in casa

Per il file `relazione.doc` viene invece richiesto uno spostamento e non una copia: il comando corretto è quindi `mv`, che sposta il file rimuovendolo dalla posizione originale:

```
mv ../../nicola/scuola/relazione.doc .
```

Lo screenshot documenta alcuni tentativi successivi: un primo mv senza indicare la destinazione (errore "missing destination file operand", perché mv richiede sempre sia la sorgente sia la destinazione), poi un mv verso la cartella corrente con il punto, e infine il comando risolutivo, mv relazione.doc seguito dal percorso relativo ../../anna/casa, eseguito con successo:



```
kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali@kali: ~/studenti/nicola/scuola
Session Actions Edit View Help

(kali@kali)~[~/studenti/nicola/scuola]
$ mv ~/studenti/nicola/scuola/relazione.doc
mv: missing destination file operand after '/home/kali/studenti/nicola/scuola/relazione.doc'
Try 'mv --help' for more information.

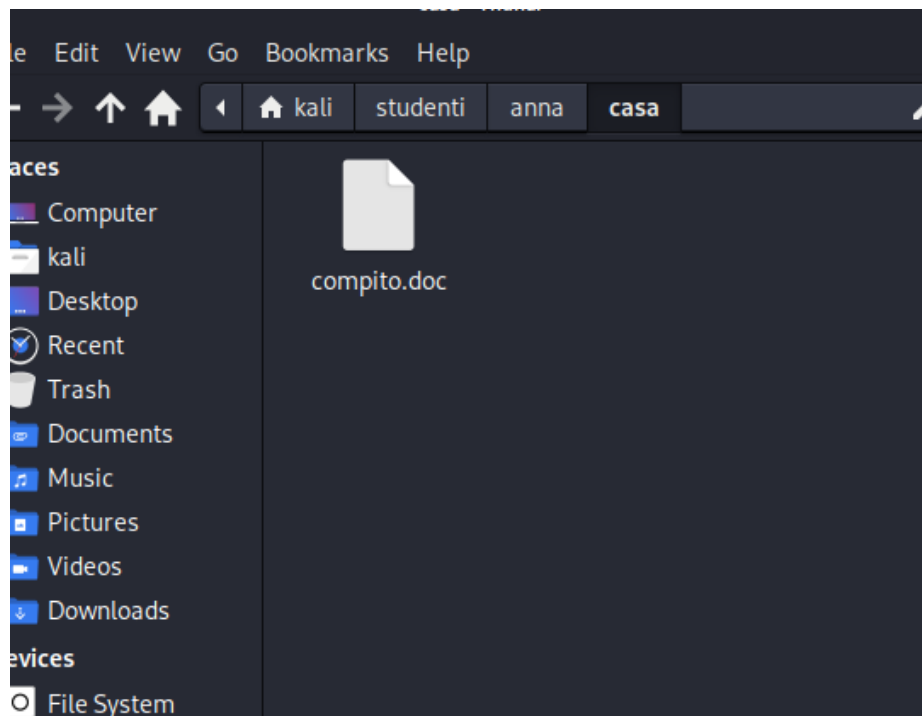
(kali@kali)~[~/studenti/nicola/scuola]
$ mv ~/studenti/nicola/scuola/relazione.doc .
mv: '/home/kali/studenti/nicola/scuola/relazione.doc' and './relazione.doc' are the same file

(kali@kali)~[~/studenti/nicola/scuola]
$ mv relazione.doc ../../anna/casa

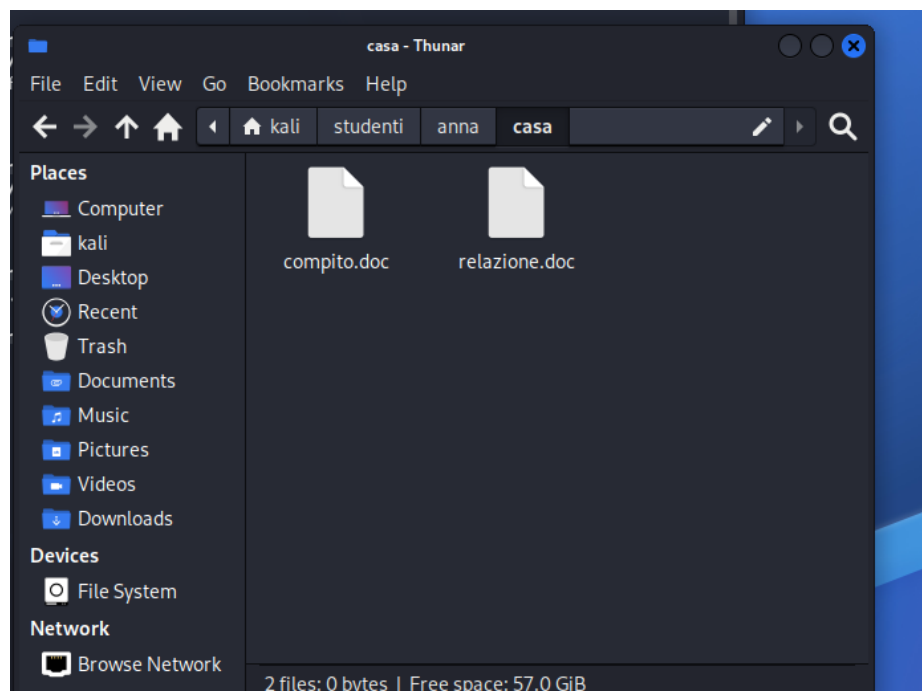
(kali@kali)~[~/studenti/nicola/scuola]
$
```

Tentativi con mv: prima manca la destinazione, poi il file viene spostato correttamente verso .../anna/casa con percorso relativo.

Le due verifiche successive in Thunar mostrano prima lo stato intermedio, con il solo compito.doc già presente in casa, poi lo stato finale con entrambi i file:



Stato della cartella casa con compito.doc già presente.



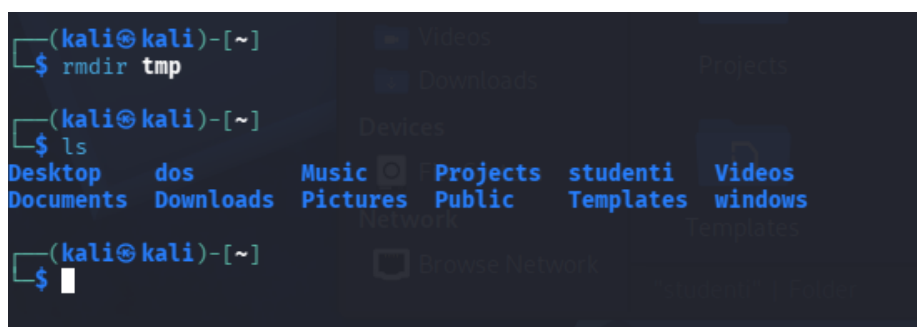
Verifica finale in Thunar: sia compito.doc sia relazione.doc sono ora presenti nella cartella casa.

Rimuovere la cartella tmp

Per cancellare una cartella si può usare `rmdir`, che però funziona solo se la cartella è vuota (in caso contrario andrebbe usato `rm -r`, che cancella ricorsivamente anche il contenuto). Essendo `tmp` vuota al momento della cancellazione, è stato sufficiente:

`rmdir tmp`

Il successivo comando `ls`, lanciato dalla HOME, conferma che `tmp` non compare più nell'elenco delle cartelle presenti:



```
(kali㉿kali)-[~]  
$ rmdir tmp  
  
(kali㉿kali)-[~]  
$ ls  
Desktop  dos      Music    Projects studenti Videos  
Documents Downloads Pictures Public  Templates windows
```

La cartella `tmp` viene rimossa con `rmdir`; il successivo `ls` conferma che non è più presente tra le cartelle della HOME.

File `pippo.txt` e permessi

Il punto d) chiede di creare il file `pippo.txt` nella cartella `lavoro`. È stato usato il comando `touch`, che crea un file vuoto (o, se il file esiste già, ne aggiorna solo la data di modifica senza toccarne il contenuto):

```
rmdir tmpcd ~/studenti/nicola/lavoro  
touch pippo.txt
```

Il punto e) chiede poi di modificare gli attributi del file rendendolo scrivibile e leggibile solo per il proprietario, e leggibile (ma non scrivibile) per tutti gli altri. I permessi in Linux si esprimono con tre cifre ottali (proprietario, gruppo, altri), dove lettura=4, scrittura=2, esecuzione=1: lettura+scrittura per il proprietario e sola lettura per tutti gli altri corrisponde quindi a 644 (6=4+2 per il proprietario, 4 per il gruppo, 4 per gli altri):

Nello screenshot vengono provate entrambe le sintassi, numerica e simbolica, con lo stesso risultato finale; il successivo `ls` conferma che il file `pippo.txt` è presente nella cartella:

```
kali@kali: ~/studenti/nicola/lavoro
Session Actions Edit View Help

(kali@kali)-[~/studenti/nicola/lavoro]
$ chmod 644 ~/studenti/nicola/lavoro/pippo.txt

(kali@kali)-[~/studenti/nicola/lavoro]
$ chmod u=rw,g=,o=r ~/studenti/nicola/lavoro/pippo.txt

(kali@kali)-[~/studenti/nicola/lavoro]
$ ls
pippo.txt

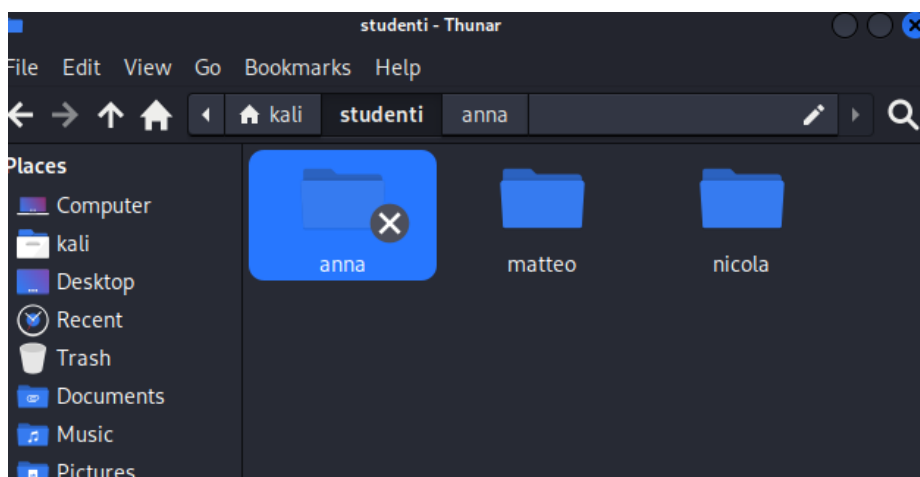
(kali@kali)-[~/studenti/nicola/lavoro]
$
```

Nascondere il contenuto della cartella anna

Il punto f: chiede di nascondere il contenuto della cartella anna. Sono state utilizzate due procedure diverse, con effetti differenti tra loro.

Prima procedura: in Linux un file o una cartella il cui nome inizia con un punto è considerato "nascosto" e non viene mostrato dai comandi/programmi in modalità standard (serve `ls -a`, o l'opzione "mostra nascosti" del file manager, per vederlo). Rinominando anna in `.anna`, la cartella smette quindi di comparire nella navigazione normale:

```
mv anna .anna
```



Questo metodo nasconde "alla vista", ma chi conosce il nome esatto può comunque accedervi normalmente. Come seconda procedura è stato quindi

usato `chmod` per togliere del tutto i permessi di accesso alla cartella (`chmod 000` = nessun permesso di lettura, scrittura o esecuzione per nessuno), rendendo il contenuto inaccessibile anche a chi ne conosce il nome:

Come si vede nello screenshot, dopo questo comando sia `ls` sia `cd` sulla cartella `anna` restituiscono "Permission denied": il contenuto risulta quindi protetto, non solo nascosto:

```
(kali㉿kali)-[~/studenti/nicola/lavoro]
$ chmod 000 ~/studenti/anna

(kali㉿kali)-[~/studenti/nicola/lavoro]
$ cd ..

(kali㉿kali)-[~/studenti/nicola]
$ cd ~

(kali㉿kali)-[~]
$ ls ~/studenti/anna
ls: cannot open directory '/home/kali/studenti/anna': Permission denied

(kali㉿kali)-[~]
$ cd ~/studenti/anna
cd: permission denied: /home/kali/studenti/anna

(kali㉿kali)-[~]
$
```

Spostarsi in lavoro e visualizzare pippo.txt

Il punto g) chiede di spostarsi nella cartella `lavoro` e visualizzare il contenuto del file `pippo.txt`. Per spostarsi si usa `cd`, mentre per visualizzare il contenuto di un file di testo il comando più diretto è `cat` (concatenate), che ne stampa a video il contenuto:

```
kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali㉿kali: ~/studenti/nicola/lavoro

Session Actions Edit View Help

(kali㉿kali)-[~]
$ cd ~/studenti/nicola/lavoro

(kali㉿kali)-[~/studenti/nicola/lavoro]
$ cat pippo.txt

(kali㉿kali)-[~/studenti/nicola/lavoro]
$ ls
pippo.txt

(kali㉿kali)-[~/studenti/nicola/lavoro]
$
```

Spostamento nella cartella lavoro e visualizzazione del contenuto di pippo.txt con cat (il file risulta vuoto, come atteso essendo stato creato con touch).

In un secondo momento è stato digitato per errore il comando "il" invece di "ls": un tipico errore di battitura, correttamente segnalato dalla shell come comando non trovato. Il successivo ls, digitato in modo corretto, conferma comunque la presenza del file pippo.txt nella cartella:

```
(kali㉿kali)-[~]  
$ cd ~/studenti/nicola/lavoro  
  
(kali㉿kali)-[~/studenti/nicola/lavoro]  
$ il  
il: command not found  
  
(kali㉿kali)-[~/studenti/nicola/lavoro]  
$ ls  
pippo.txt  
  
(kali㉿kali)-[~/studenti/nicola/lavoro]  
$
```

Rimozione di amici e di tutte le cartelle

Il punto h) chiede di rimuovere la cartella amici (sotto matteo). Essendo vuota, è sufficiente rmdir, che come già visto al punto c) funziona solo su cartelle senza contenuto:

```
rmdir ~/studenti/matteo/amici
```

Il punto i), infine, chiede di rimuovere tutte le cartelle create in precedenza. A questo punto le cartelle contengono però dei file al loro interno (compito.doc, relazione.doc, pippo.txt, ecc.), quindi rmdir non sarebbe sufficiente perché richiede cartelle vuote: è necessario usare rm con l'opzione -r (recursive), che cancella ricorsivamente le cartelle e tutto il loro contenuto:

```
cd ~  
rm -r dos windows studenti
```

La cartella tmp non compare in questo comando perché era già stata rimossa in precedenza al punto c). Con questi comandi la HOME torna alla situazione di partenza, priva delle cartelle create appositamente per l'esercizio.

Conclusione

Attraverso questo esercizio sono stati messi in pratica i comandi fondamentali della shell Linux per la navigazione e la gestione del filesystem:

- `cd`, `mkdir`, `rmdir` per navigare e creare/rimuovere cartelle;
- `cp` e `mv` per copiare e spostare file, con percorsi sia relativi sia assoluti;
- `rm -r` per rimuovere ricorsivamente cartelle non vuote;
- `touch` e `cat` per creare e visualizzare il contenuto di file di testo;
- `chmod`, in forma numerica e simbolica, per gestire i permessi di lettura/scrittura di file e cartelle;
- due tecniche diverse per "nascondere" una cartella: rinominarla con un punto iniziale (nascosta alla vista) oppure togliere i permessi con `chmod 000` (nascosta e protetta dall'accesso).

Il lavoro pratico ha inoltre evidenziato l'importanza della posizione corrente nella shell quando si usano percorsi relativi, e la necessità di prestare attenzione alla sintassi esatta dei comandi (ad esempio la differenza tra `mkdir`, usato per creare cartelle, e `touch`, usato per creare file).